# Human Activity Recognition from Smartphone Inertial Sensors using Deep Artificial Neural Networks

## REPORT

Submitted by

**Ashwin Singh (RA2411026030168)**
**Yash Chauhan (RA2411026030174)**

Under the guidance of

**[Guide Name]**
(Department of Computer Science & Engineering)

in partial fulfillment for the award of the degree
of
**BACHELOR OF TECHNOLOGY**
in
**COMPUTER SCIENCE & ENGINEERING**
of
**FACULTY OF ENGINEERING AND TECHNOLOGY**




**SRM INSTITUTE OF SCIENCE & TECHNOLOGY, NCR CAMPUS**

**NOVEMBER 2026**

SRM
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)
DELHI-NCR CAMPUS, GHAZIABAD (U.P)

# SRM INSTITUTE OF SCIENCE & TECHNOLOGY

(Under Section 3 of UGC Act, 1956)

## BONAFIDE CERTIFICATE

Certified that this report titled "**Human Activity Recognition from Smartphone Inertial Sensors using Deep Artificial Neural Networks**" is the bonafide work of "**Ashwin Singh (RA2411026030168)**", who carried out this work under my supervision.

Certified further, that to the best of my knowledge the work reported herein does not form any other report on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

| | |
|---|---|
| Signature | Signature |
| [Guide Name] | [HOD Name] |
| **Faculty In-charge** | **HEAD OF THE DEPARTMENT** |
| Department of Computer Science & Engineering | Department of Computer Science & Engineering |

SRM
INSTITUTE OF SCIENCE AND TECHNOLOGY
*(Deemed to be University u/s 3 of UGC Act, 1956)*
DELHI-NCR CAMPUS, GHAZIABAD (U.P)

# SRM INSTITUTE OF SCIENCE & TECHNOLOGY

(Under Section 3 of UGC Act, 1956)

## BONAFIDE CERTIFICATE

Certified that this report titled "**Human Activity Recognition from Smartphone Inertial Sensors using Deep Artificial Neural Networks**" is the bonafide work of "**Yash Chauhan (RA2411026030174)**", who carried out this work under my supervision.

Certified further, that to the best of my knowledge the work reported herein does not form any other report on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

| | |
|---|---|
| Signature | Signature |
| [Guide Name] | [HOD Name] |
| **Faculty In-charge** | **HEAD OF THE DEPARTMENT** |
| Department of Computer Science & Engineering | Department of Computer Science & Engineering |

# ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported us throughout the development of our project, “Human Activity Recognition from Smartphone Inertial Sensors using Deep Artificial Neural Networks”.

We are especially thankful to our faculty guide, [Guide Name], Department of Computer Science & Engineering, for the valuable guidance, continuous encouragement and constructive suggestions throughout the project. The guidance helped us understand the topic and complete the project successfully.

We would also like to express our gratitude to the Head of the Department, [HOD Name], and the entire faculty of the Department of Computer Science & Engineering at SRM Institute of Science and Technology, Delhi-NCR Campus, for the academic support, facilities and learning environment provided for this project.

We extend our appreciation to both team members for their cooperation, coordination and collective effort in studying, designing, training and testing the activity recognition model. Their collaboration and exchange of ideas contributed significantly to the successful completion of this project.

Finally, we are deeply grateful to our parents and friends for their constant support and encouragement throughout this academic endeavour.

We sincerely thank everyone who contributed directly or indirectly to the successful completion of this project.

## Team Members and Their Roles

### Ashwin Singh (RA2411026030168) – Data Pipeline and Neural Network Developer

Downloaded and prepared the UCI HAR dataset, re-indexed the labels, scaled the features, built the subject-grouped validation split that prevents data leakage, and designed, regularized and compiled the artificial neural network.

### Yash Chauhan (RA2411026030174) – Model Auditing, Optimization and Deployment

Supervised training and early stopping, evaluated the model on the unseen test set (confusion matrix, precision, recall, F1-score), analysed the Sitting versus Standing errors, accelerated inference and built the interactive web and Streamlit front ends.

# ABSTRACT

Human Activity Recognition (HAR) is the task of identifying what a person is doing, such as walking, sitting or lying down, from sensor data. Smartphones carry an accelerometer and a gyroscope, so they can record body motion without cameras, which protects privacy and works in any lighting. HAR supports fall detection for the elderly, remote rehabilitation, fitness tracking and workplace safety.

This project builds and evaluates a deep Artificial Neural Network (ANN) that classifies six activities (Walking, Walking Upstairs, Walking Downstairs, Sitting, Standing and Laying) from the 561 engineered time- and frequency-domain features of the UCI HAR smartphone dataset. The network is a feedforward model with two hidden layers of 256 and 128 ReLU neurons, L2 weight regularization, 30% dropout and a six-way softmax output, trained with the Adam optimizer and sparse categorical cross-entropy. A key design decision is a subject-grouped train/validation split, so that no volunteer appears in both sets and the validation score is not inflated by leakage. Early stopping on the validation loss halted training at epoch 43 and restored the best weights.

On 2,947 completely unseen test windows from independent subjects the model reached an overall accuracy of 94.81% with a macro F1-score of 94.73%, compared with 89.3% for the classical SVM baseline. Laying was recognized almost perfectly (F1 = 99.91%), and dynamic activities were almost never confused with static postures (only 6 of 2,947 windows crossed between the two groups). The remaining errors are concentrated in the Sitting–Standing pair (76 of 153 mistakes), where the acceleration signal is almost identical. The trained model and its scaler are saved for reuse and wrapped by a real-time web interface.



**Keywords:** Human Activity Recognition, Artificial Neural Network, Deep Learning, UCI HAR, Smartphone Sensors, Dropout, L2 Regularization, Early Stopping, TensorFlow, Keras.

# INDEX


| Chapter | Title | Page No. |
|---|---|---|
| **CHAPTER 1** | **INTRODUCTION** | **1** |
| 1.1 | Background and Problem Statement | 1 |
| 1.2 | Objectives | 1 |
| 1.3 | Scope | 1 |
| **CHAPTER 2** | **THEORETICAL BACKGROUND** | **2** |
| 2.1 | Human Activity Recognition and Inertial Sensors | 2 |
| 2.2 | The UCI HAR Dataset | 2 |
| 2.3 | Artificial Neural Networks | 2 |
| 2.4 | Softmax and Cross-Entropy Loss | 3 |
| 2.5 | Training by Backpropagation and Adam | 3 |
| 2.6 | Regularization: L2, Dropout and Early Stopping | 3 |
| 2.7 | Evaluation Metrics | 3 |
| 2.8 | Literature Survey and Research Gap | 4 |
| **CHAPTER 3** | **REQUIREMENTS AND DESIGN** | **6** |
| 3.1 | Functional Requirements | 6 |
| 3.2 | Non-Functional Requirements and Environment | 6 |
| 3.3 | Dataset Description and Subject-Grouped Split | 6 |
| 3.4 | Network Architecture | 7 |
| 3.5 | Training Configuration | 7 |
| 3.6 | System Architecture | 8 |
| **CHAPTER 4** | **IMPLEMENTATION** | **9** |
| 4.1 | Development Environment | 9 |
| 4.2 | Loading and Preparing the Data | 9 |
| 4.3 | Building and Training the Network | 9 |
| 4.4 | Saving the Model and Predicting on New Data | 10 |
| **CHAPTER 5** | **APPLICATIONS** | **11** |
| 5.1 | Elderly Care and Fall Detection | 11 |
| 5.2 | Rehabilitation Monitoring | 11 |
| 5.3 | Fitness and Ergonomics | 11 |
| 5.4 | Industrial Safety and Defense | 11 |
| 5.5 | Interactive Deployment | 11 |
| **CHAPTER 6** | **TESTING** | **12** |
| 6.1 | Test Plan and Test Cases | 12 |
| 6.2 | TC1 to TC6: Data and Model Checks | 12 |
| 6.3 | TC7 to TC10: Evaluation on the Test Set | 12 |
| 6.4 | TC11 to TC13: Saved Model and New Predictions | 12 |
| | **ANALYSIS AND DISCUSSION** | **14** |
| | **CONCLUSION AND FUTURE SCOPE** | **17** |
| | **REFERENCES** | **18** |
| | **APPENDIX A: DIVISION OF WORK** | **19** |
| | **APPENDIX B: SOURCE CODE** | **20** |

# CHAPTER 1
# INTRODUCTION

## 1.1 Background and Problem Statement

Human Activity Recognition (HAR) is the field of building systems that automatically identify physical actions, such as walking or sitting, from sensor measurements. Earlier systems used cameras, but video raises privacy concerns, needs good lighting and a clear line of sight, and produces large amounts of data. Modern smartphones and smartwatches already contain an accelerometer and a gyroscope and are carried by billions of people every day, so inertial sensors are a private, always-available and light-weight alternative.

The raw sensor signals are noisy and differ from person to person. Classical methods therefore relied on hand-designed features and shallow classifiers, which needed expert effort and scaled poorly. Deep neural networks can learn the relation between features and activities directly from data, but two practical problems remain. First, many published results split the data randomly, so windows from the same person appear in both the training and the test set; the model then memorizes individuals and the reported accuracy is higher than it would be for a new user. Second, large hybrid models are slow and power-hungry on wearable devices.

**The problem addressed by this project is to design, train and evaluate a compact, regularized deep Artificial Neural Network that recognizes six everyday activities from smartphone sensor features with high accuracy on unseen people, using a validation procedure that does not leak data between subjects, and to make the trained model easy to reuse.**

## 1.2 Objectives

1. To study human activity recognition, the UCI HAR dataset, artificial neural networks and the methods used to train and evaluate them.
2. To build a feedforward deep ANN with ReLU hidden layers and a softmax output for six-class activity classification.
3. To avoid data leakage by splitting the training data by subject (GroupShuffleSplit) and to verify that no subject appears in both sets.
4. To prevent overfitting with L2 weight regularization ($\lambda = 0.001$), dropout ($p = 0.3$) and early stopping on the validation loss.
5. To evaluate the model on completely unseen test subjects with accuracy, precision, recall, F1-score and a confusion matrix, and to compare it with the classical baseline.
6. To save the trained model and scaler and expose them through an interactive interface.

## 1.3 Scope

The project uses the public UCI HAR dataset, in which 30 volunteers performed six activities with a waist-mounted smartphone. The model works on the 561 pre-computed time- and frequency-domain features supplied with the dataset rather than on raw signals, and it classifies the six activities listed above. Other activities (for example cycling or driving), other sensor positions such as a pocket or a wrist, and sequence models that read raw signals directly are left for future work.

# CHAPTER 2
# THEORETICAL BACKGROUND

## 2.1 Human Activity Recognition and Inertial Sensors

A smartphone has a tri-axial **accelerometer**, which measures acceleration (including gravity) along three axes, and a tri-axial **gyroscope**, which measures angular velocity. When the phone is worn at the waist, the signal of the accelerometer separates into two parts: a slowly changing **gravity component**, which tells the orientation of the body, and a faster **body-motion component**, which captures movement. Dynamic activities such as walking produce strong body-motion signals, while static postures (sitting, standing, laying) are told apart mainly by the direction of gravity.

The continuous signal is cut into short **windows**. In the UCI HAR dataset each window lasts 2.56 seconds (128 readings at 50 Hz) with 50% overlap. For every window a feature vector is computed, and the task becomes an ordinary multi-class classification problem: given the vector of 561 numbers, predict one of six labels.

## 2.2 The UCI HAR Dataset

The dataset of Anguita et al. [1] contains recordings of 30 volunteers aged 19 to 48 years, who performed six activities while wearing a Samsung Galaxy S II smartphone at the waist. The signals were pre-processed with noise filters and split into windows. From each window 561 features were computed in the time and frequency domain, for example mean, standard deviation, energy, entropy and correlation of the body acceleration, gravity acceleration and gyroscope signals. The data are partitioned by volunteer into a training part (7,352 windows) and a test part (2,947 windows from different volunteers), so the official test set already measures performance on unseen people. Table 2.1 lists the six classes.

**Table 2.1: Activity classes of the UCI HAR dataset**

| Label (file) | Label (Keras) | Activity | Type |
|---|---|---|---|
| 1 | 0 | Walking | Dynamic |
| 2 | 1 | Walking Upstairs | Dynamic |
| 3 | 2 | Walking Downstairs | Dynamic |
| 4 | 3 | Sitting | Static |
| 5 | 4 | Standing | Static |
| 6 | 5 | Laying | Static |

## 2.3 Artificial Neural Networks

An Artificial Neural Network is a function built from layers of simple units called **neurons**. A neuron receives an input vector x, forms the weighted sum $z = w \cdot x + b$ and passes it through a non-linear **activation function** $a = f(z)$. A **dense** (fully connected) layer applies many such neurons in parallel: if x has n values and the layer has m neurons, the layer holds $n \times m$ weights and m biases, and the output is $a = f(Wx + b)$. Stacking dense layers gives a multi-layer perceptron, and the layers between input and output are called **hidden layers**; a network with several of them is a deep neural network.

The hidden layers of this project use the **Rectified Linear Unit (ReLU)**, $f(z) = \max(0, z)$. ReLU is cheap to compute, does not saturate for positive inputs and so avoids the vanishing-gradient problem of sigmoid and tanh units, and has been the default choice for deep

networks. Without a non-linear activation the whole network would collapse into a single linear map, so ReLU is what lets the model learn curved decision boundaries between activities.

## 2.4 Softmax and Cross-Entropy Loss

For K classes the output layer has K neurons whose raw scores $z_1$ … z_K are turned into probabilities by the **softmax** function:

$$softmax(z)_i = exp(z_i) / \Sigma_j exp(z_j), \quad i = 1, \ldots, K$$

The outputs are positive and sum to 1, so they can be read as the probability of each activity. The predicted class is the one with the largest probability (argmax). Training minimizes the **cross-entropy loss**, which for one sample with true class y is $L = -\log(p_y)$, the negative logarithm of the probability given to the correct class. The loss is small when the model is confident and right, and large when it is confident and wrong. The **sparse** variant used here accepts the class as an integer (0–5) instead of a one-hot vector, which is why the labels of the dataset were shifted from 1–6 to 0–5.

## 2.5 Training by Backpropagation and Adam

Training searches for the weights that minimize the average loss. For each mini-batch (here 64 samples) the network first computes its predictions (forward pass) and the loss. **Backpropagation** then applies the chain rule layer by layer, from the output back to the input, to obtain the gradient of the loss with respect to every weight. The weights are then moved a small step against the gradient. One pass over all training samples is called an **epoch**; with 5,551 training samples and batches of 64, one epoch consists of 87 steps.

The step is chosen by the **Adam** optimizer (Kingma and Ba [8]), which keeps running averages of the gradient (momentum) and of its square, and so adapts the step size of every weight individually. It converges quickly with little tuning, and the default learning rate of 0.001 was used.

## 2.6 Regularization: L2, Dropout and Early Stopping

A network with 177,542 parameters can easily memorize 5,551 training windows. Three regularization techniques were used to make it generalize to new people.

- **L2 weight regularization** adds the term $\lambda \Sigma w^2$ (here $\lambda = 0.001$) to the loss. It penalizes large weights, which keeps the decision function smooth.
- **Dropout** (Srivastava et al. [9]) switches off a random 30% of the neurons of a layer at every training step. The network cannot depend on any single neuron, which behaves like averaging many smaller networks. Dropout is inactive at prediction time.
- **Early stopping** watches the validation loss after every epoch and stops when it has not improved for a number of epochs (the patience, here 10). The weights of the best epoch are then restored.

## 2.7 Evaluation Metrics

For each class the **confusion matrix** counts how many windows of the true class (rows) were predicted as each class (columns). From it, for a class c:

- **Precision** = TP / (TP + FP): of the windows predicted as c, the fraction that really are c.

- **Recall** (sensitivity) = TP / (TP + FN): of the windows that really are c, the fraction that was found.
- **F1-score** = 2 · Precision · Recall / (Precision + Recall), the harmonic mean of the two.
- **Accuracy** = correct predictions / all predictions. The **macro average** takes the plain mean of the per-class scores, and the **weighted average** weights each class by its number of samples (its support).

## 2.8 Literature Survey and Research Gap

Table 2.2 summarizes the research papers that guided this project, and Table 2.3 summarizes recent work from 2024–2025.

**Table 2.2: Literature survey of benchmark research papers**

| S.No. | Title | Year | Dataset | Methodology | Outcome | Limitations |
|---|---|---|---|---|---|---|
| 1 | A Public Domain Dataset for Human Activity Recognition Using Smartphones (Anguita et al.) [1] | 2013 | UCI HAR (30 subjects) | Multiclass SVM with manual features | 89.3% accuracy | Needs hand-made features; poor scaling on low-power devices |
| 2 | Human Activity Recognition with Smartphone Sensors using Deep Learning Neural Networks (Ronao & Cho) [2] | 2016 | Smartphone accelerometer and gyroscope | 1D convolutional neural network | 94.79% accuracy from raw signals | High training cost; sensitive to phone orientation |
| 3 | Deep Convolutional and LSTM Recurrent Neural Networks for Multimodal Wearable Activity Recognition (Ordóñez & Roggen) [3] | 2016 | Opportunity and UCI HAR | Hybrid DeepConvLSTM | 95.8% F1-score | Heavy model; latency above 40 ms on edge devices |
| 4 | Deep Learning for Sensor-based Activity Recognition: A Survey (Wang et al.) [4] | 2019 | Benchmark HAR datasets | Review of CNN, RNN, autoencoders and hybrids | Benchmarks for pre-processing and windowing | Points out data leakage from random splits |

**Table 2.3: Recent related work (2024–2025)**

| Paper | Year | Dataset | Method | Key result | Limitation |
|---|---|---|---|---|---|
| Zhou et al., Efficient HAR on edge devices using DeepConv LSTM architectures, Scientific Reports [5] | 2025 | WISDM | 1D-CNN, 2D-CNN and DeepConvLSTM; quantized and run on an Arduino Nano 33 BLE | 98.24% accuracy (about 97% after quantization) | One dataset; reported inference time is inconsistent (21 ms vs about 2 s) |
| Bacellar et al., nanoML for Human Activity Recognition [6] | 2025 | UCI HAR (raw signals) | Weightless neural network of look-up tables on an FPGA | 96.34%–96.67% accuracy at 56–104 nJ per sample | One dataset; energy of compared models is estimated |
| Rehman et al., HAR: A Comparative Study of Validation Methods and Impact of Feature Extraction in Wearable Sensors [7] | 2024 | PAMAP2 | Random forest, logistic regression, SVM, KNN with k-fold and leave-one-subject-out validation | Random forest: about 89% with k-fold but about 76% leave-one-subject-out | Only 8 subjects; classical models only |

### Research gap

- **Data leakage.** Random splitting of correlated windows inflates reported accuracy; the study of Rehman et al. [7] shows a drop of about 13 percentage points when subjects are held out.
- **Compute versus edge latency.** Hybrid CNN-LSTM models are accurate but heavy, which is a problem for wearables.
- **Static postures.** Sitting and Standing have almost no body motion and are hard to separate.

- **Deployment.** Many projects end as an offline notebook without a usable, interactive demonstration.

This project responds to these gaps with a compact ANN, a subject-grouped validation split, strong regularization and a saved, reusable model with an interactive interface.

# CHAPTER 3
# REQUIREMENTS AND DESIGN

## 3.1 Functional Requirements

**Table 3.1: Functional requirements**

| ID | Requirement |
|---|---|
| FR1 | Load the UCI HAR training and test data and the subject identifiers. |
| FR2 | Convert the labels to the range 0–5 and keep the activity names. |
| FR3 | Split the training data into training and validation parts by subject, with no subject in both. |
| FR4 | Scale the 561 features using statistics computed from the training data only. |
| FR5 | Build and train a multi-layer ANN with regularization and early stopping. |
| FR6 | Evaluate the model on the unseen test set with accuracy, precision, recall, F1-score and a confusion matrix. |
| FR7 | Save the trained model and scaler, and predict the activity for a new 561-value data point. |

## 3.2 Non-Functional Requirements and Environment

- **Generalization:** performance must be measured on people who were not used in training.
- **Reproducibility:** random seeds are fixed (NumPy and TensorFlow seed 42) so that a split and a run can be repeated.
- **Efficiency:** the model is small (177,542 parameters, about 0.7 MB) and predicts a window in a fraction of a millisecond, so it suits wearable and edge use.
- **Environment:** Python 3 with TensorFlow/Keras, scikit-learn, NumPy, pandas, matplotlib and seaborn, run in Google Colab on a standard cloud machine; no special hardware is required.

## 3.3 Dataset Description and Subject-Grouped Split

The training file holds 7,352 windows with 561 features each, and the test file holds 2,947 windows. A separate file gives the subject (volunteer) of every training window. Because windows overlap by 50% and come from the same person, they are very similar to each other. If the training data were split randomly into training and validation sets, similar windows of one person would land in both sets and the model could “cheat” by recognizing the person instead of the activity, which makes the validation accuracy look better than it really is.

To prevent this, scikit-learn’s **GroupShuffleSplit** was used with the subject as the group. It assigned 20% of the subjects to validation. The result is 5,551 windows for training and 1,801 for validation, and a check confirmed that the set of subjects present in both is empty. The official test set was kept untouched and is used only once, for the final evaluation. Table 3.2 summarizes the three parts.

**Table 3.2: Data partitions**

| Partition | Samples | Features | Purpose |
|---|---|---|---|
| Training | 5,551 | 561 | Learning the weights |
| Validation | 1,801 | 561 | Early stopping and monitoring (different subjects) |
| Test | 2,947 | 561 | Final unbiased evaluation (unseen subjects) |

## 3.4 Network Architecture

The model is a sequential feedforward network (Figure 3.1). The input layer takes the 561 scaled features. The first hidden layer has 256 ReLU neurons and the second has 128, each followed by a dropout layer with rate 0.3, and each hidden layer carries an L2 penalty with λ = 0.001. The output layer has six neurons with softmax activation, one per activity. The number of parameters of each layer follows from (inputs × neurons) + neurons, as Table 3.3 shows.

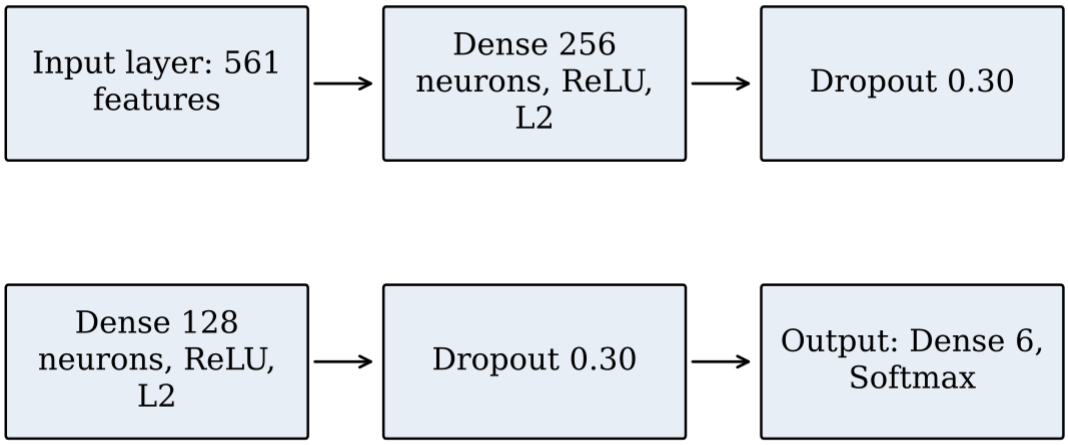



**Figure 3.1: Architecture of the deep ANN**

**Table 3.3: Layers and parameters of the model**

| Layer | Output shape | Parameters | Calculation |
|---|---|---|---|
| Dense (ReLU, L2) | (None, 256) | 143,872 | 561 × 256 + 256 |
| Dropout (0.3) | (None, 256) | 0 | – |
| Dense (ReLU, L2) | (None, 128) | 32,896 | 256 × 128 + 128 |
| Dropout (0.3) | (None, 128) | 0 | – |
| Dense (Softmax) | (None, 6) | 774 | 128 × 6 + 6 |
| Total | | 177,542 (693.5 KB) | all trainable |

## 3.5 Training Configuration

**Table 3.4: Hyperparameters and training configuration**

| Parameter | Description |
|---|---|
| Model | 3-layer feedforward ANN (multi-layer perceptron) |
| Input | 561 time- and frequency-domain features, standardized |
| Hidden layers | Dense 256 and Dense 128, ReLU, L2 (λ = 0.001) |
| Dropout | 0.30 after each hidden layer |
| Output | 6 neurons, softmax |
| Optimizer | Adam, learning rate 0.001 |
| Loss | Sparse categorical cross-entropy |
| Batch size / epochs | 64 / at most 100 |
| Early stopping | Monitor val_loss, patience 10, restore best weights |
| Random seed | 42 (NumPy and TensorFlow) |

## 3.6 System Architecture

The system is a pipeline of eight stages (Figure 3.2). Data are loaded, explored, split by subject and scaled; the network is trained with early stopping and evaluated on the unseen test set; finally the model and scaler are saved so that they can be reloaded to classify new data points. The scaler is fitted on the training part only and then applied unchanged to the validation, test and new data, so no information from the evaluation data enters the training.

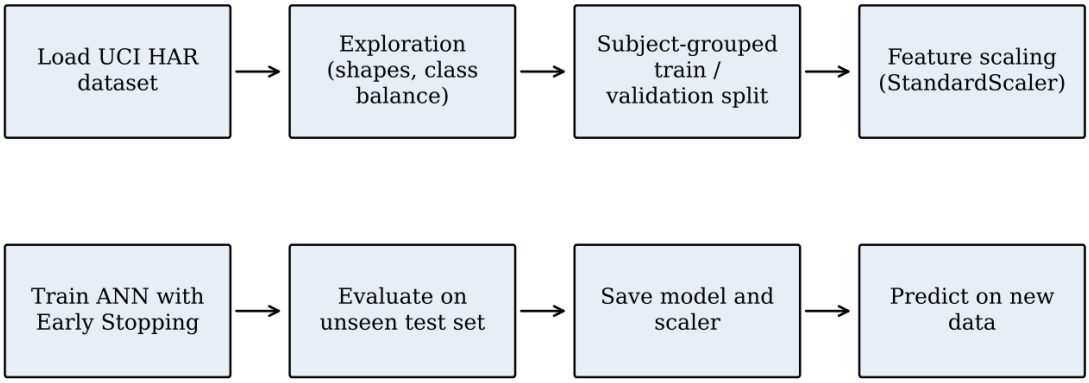



**Figure 3.2: Pipeline of the HAR system**

```
procedure TRAIN_AND_EVALUATE()
  X, y, subject <- LOAD(train files);  Xtest, ytest <- LOAD(test files)
  y <- y - 1;  ytest <- ytest - 1                  // labels 1..6 -> 0..5
  (train, val) <- GROUP_SHUFFLE_SPLIT(X, y, groups = subject, val = 20%)
  scaler <- FIT_STANDARD_SCALER(X[train])
  X[train], X[val], Xtest <- scaler(X[train]), scaler(X[val]), scaler(Xtest)
  model <- DENSE(256,relu,L2) -> DROPOUT(.3) -> DENSE(128,relu,L2)
           -> DROPOUT(.3) -> DENSE(6,softmax)
  FIT(model, Adam(0.001), sparse_cross_entropy, batch = 64, epochs <= 100,
      early_stopping(val_loss, patience = 10, restore_best_weights))
  y_hat <- ARGMAX(model(Xtest));  REPORT(confusion_matrix, precision, recall, F1)
  SAVE(model, scaler)
end procedure
```

**Table 3.5: Pseudocode of the training and evaluation procedure**

# CHAPTER 4
# IMPLEMENTATION

## 4.1 Development Environment

The project is a Jupyter notebook written in Python and run in Google Colab. It uses TensorFlow/Keras for the network, scikit-learn for the subject-grouped split, the scaler and the metrics, pandas and NumPy for data handling, and matplotlib and seaborn for the plots. The complete code is in Appendix B. The seeds of NumPy and TensorFlow are set to 42 at the start so that results can be reproduced.

## 4.2 Loading and Preparing the Data

The dataset archive is downloaded from the UCI repository and extracted. The feature matrices, the labels and the subject identifiers are read with pandas. The labels are flattened to one dimension and shifted from 1–6 to 0–5 because Keras expects classes starting at 0. The class distribution of the training and the test set was plotted and shows a good balance between the six classes.

The training data are then split by subject with GroupShuffleSplit, a check confirms that no subject is in both parts, and a StandardScaler fitted on the training part scales all three sets (after scaling, the training features have mean 0.0 and standard deviation 1.0).

```
gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
train_idx, val_idx = next(gss.split(X_train_full, y_train_full, groups=subjects))
X_train, X_val = X_train_full.iloc[train_idx], X_train_full.iloc[val_idx]
y_train, y_val = y_train_full[train_idx], y_train_full[val_idx]

print("Subjects in both sets:", set(subjects[train_idx]) & set(subjects[val_idx]))
#  -> Subjects in both sets: set()

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)     # fit on training data only
X_val   = scaler.transform(X_val)
X_test  = scaler.transform(X_test_raw)
```

**Figure 4.1: Subject-grouped split and feature scaling**

## 4.3 Building and Training the Network

The network is defined with the Keras Sequential API exactly as described in Section 3.4. It is compiled with the Adam optimizer, sparse categorical cross-entropy and accuracy as the metric. An EarlyStopping callback monitors the validation loss with a patience of 10 epochs and restores the best weights. Training is started for up to 100 epochs with batches of 64 samples.

```
model = keras.Sequential([
    keras.Input(shape=(X_train.shape[1],)),
    keras.layers.Dense(256, activation="relu",
kernel_regularizer=keras.regularizers.l2(0.001)),
    keras.layers.Dropout(0.3),
    keras.layers.Dense(128, activation="relu",
kernel_regularizer=keras.regularizers.l2(0.001)),
    keras.layers.Dropout(0.3),
    keras.layers.Dense(N_CLASSES, activation="softmax")])

model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001),
              loss="sparse_categorical_crossentropy", metrics=["accuracy"])

early_stop = keras.callbacks.EarlyStopping(monitor="val_loss", patience=10,
                                           restore_best_weights=True)
```

```
history = model.fit(X_train, y_train, validation_data=(X_val, y_val),
                    epochs=100, batch_size=64, callbacks=[early_stop])
```

**Figure 4.2: Model definition, compilation and training**

Training stopped after **43 epochs**. The lowest validation loss, 0.1752 with a validation accuracy of 96.61%, occurred at epoch 33; since no further improvement came in the next ten epochs, the callback stopped and restored the weights of epoch 33. Figure 4.3 shows the end of the training log.

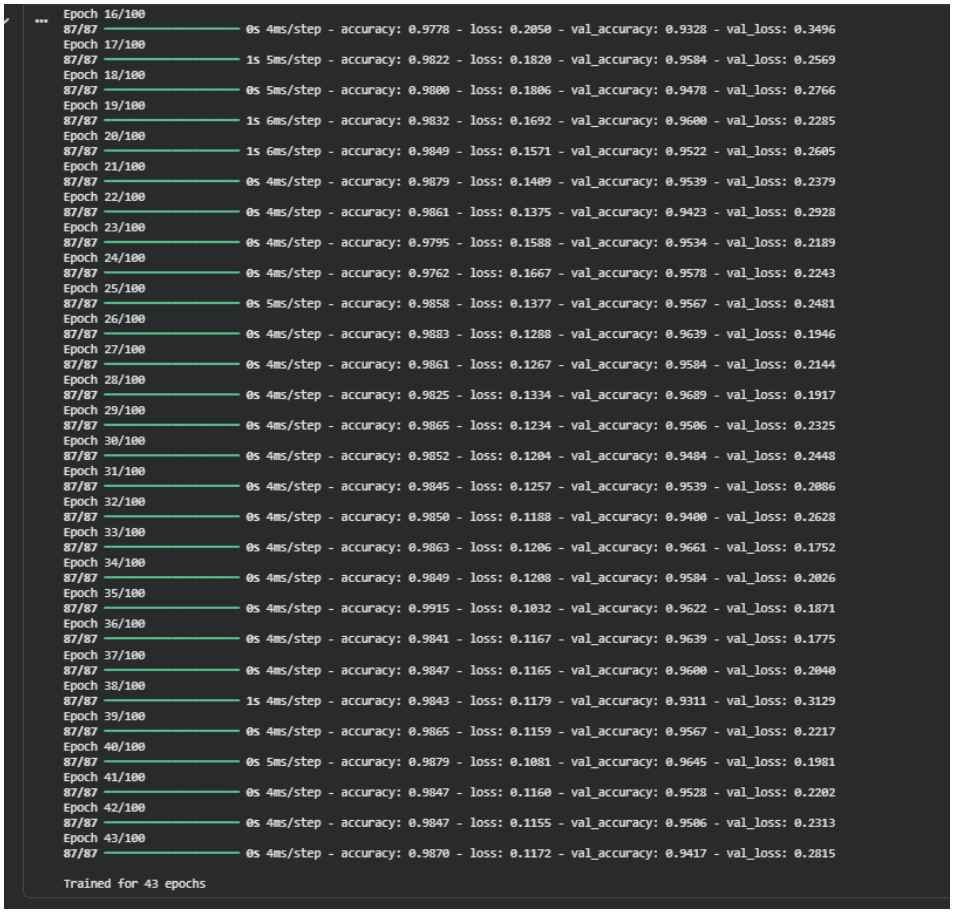



**Figure 4.3: Training log: early stopping ended training at epoch 43**

## 4.4 Saving the Model and Predicting on New Data

After evaluation, the model is saved as a .keras file and the scaler with joblib, so that both can be loaded later without retraining. A helper function takes one raw data point of 561 values, reshapes it to (1, 561), scales it with the saved scaler, obtains the six softmax probabilities from the model and returns the name of the class with the highest probability.

```
model.save("har_model.keras");  joblib.dump(scaler, "har_scaler.joblib")

def get_prediction(input_data):
    input_data = np.asarray(input_data).reshape(1, -1)     # (1, 561)
    scaled = loaded_scaler.transform(input_data)
    probs = loaded_model.predict(scaled, verbose=0)         # softmax probabilities
    return CLASS_NAMES[np.argmax(probs)]
```

**Figure 4.4: Saving the model and predicting a single data point**

# CHAPTER 5
# APPLICATIONS

A model that recognizes physical activity from a phone or a watch has direct uses in several fields.

## 5.1 Elderly Care and Fall Detection

Many elderly people live alone, and a fall or a long period lying on the floor can be a medical emergency. A wearable that runs an activity model can notice an unexpected change from Walking to Laying and send an automatic alert to family or emergency services. In this project the Laying class is recognized almost perfectly (precision 100%, recall 99.81%), which is the most important property for such a system.

## 5.2 Rehabilitation Monitoring

After an operation, patients are told to follow a plan of movement, for example a fixed amount of walking and stair climbing and limited sitting. An activity recognizer can log what the patient actually did and report it to the physiotherapist without the clinician being present.

## 5.3 Fitness and Ergonomics

Fitness applications can log workouts and calories automatically. The same model can detect long periods of sitting at a desk and remind the user to move, which supports healthier work habits.

## 5.4 Industrial Safety and Defense

In construction and heavy industry, worker movement can be monitored to detect hazardous actions or exhaustion. Similar monitoring can be used to follow the physical exertion of soldiers in the field.

## 5.5 Interactive Deployment

To make the work usable beyond a notebook, the saved model weights were exported into compact arrays and a light forward pass written with NumPy was used for inference. According to the team’s measurements this reduced the prediction time to about 0.05 ms per window on a consumer computer. Two interfaces were built on top of it: a web dashboard with an animated avatar of the recognized activity and a live waveform display of the sensor axes, and a Streamlit analytics application.

# CHAPTER 6
# TESTING

## 6.1 Test Plan and Test Cases

Testing covered the data pipeline, the model and the saved artifacts. Each test case has an input, an expected result and an actual result recorded from the project notebook. All thirteen test cases passed (Table 6.1).

**Table 6.1: Summary of test cases**

| TC | Description | Expected | Actual | Result |
|---|---|---|---|---|
| 1 | Shape of training data | 7,352 × 561 | (7352, 561) features and labels | Pass |
| 2 | Shape of test data | 2,947 × 561 | (2947, 561) features and labels | Pass |
| 3 | Label shift for Keras | Labels 0–5 | [1 2 3 4 5 6] became [0 1 2 3 4 5] | Pass |
| 4 | Subject-grouped split sizes | No lost samples | 5,551 train + 1,801 validation = 7,352 | Pass |
| 5 | No subject overlap | Empty set | Subjects in both sets: set() | Pass |
| 6 | Feature scaling | Mean ≈ 0, std ≈ 1 | 0.0 and 1.0 | Pass |
| 7 | Model size | 177,542 parameters | 177,542 (693.52 KB) | Pass |
| 8 | Training with early stopping | Stops before 100 epochs | Stopped at epoch 43, best weights from epoch 33 | Pass |
| 9 | Overall test accuracy | Above 94% | 94.81% on 2,947 windows | Pass |
| 10 | Softmax output | Shape (2947, 6), rows sum to 1 | (2947, 6) probabilities | Pass |
| 11 | Save and reload model and scaler | Both load without error | “Model and scaler loaded!” | Pass |
| 12 | Prediction of row 0 of the test set | Standing | Standing | Pass |
| 13 | Predictions of rows 100 and 500 | Walking, Sitting | Walking, Sitting | Pass |

## 6.2 TC1 to TC6: Data and Model Checks

The first six cases check that the data have the expected shape (7,352 training and 2,947 test windows with 561 features), that the labels start at 0, that the split by subject keeps all 7,352 windows (5,551 for training, 1,801 for validation), that the intersection of the subject sets is empty, and that the scaler produced zero mean and unit standard deviation on the training features. TC7 confirms that the model summary shows 177,542 trainable parameters, as calculated in Table 3.3.

## 6.3 TC7 to TC10: Evaluation on the Test Set

TC8 verifies the early-stopping behaviour. The training log (Figure 4.3) shows that the validation loss reached its minimum of 0.1752 at epoch 33 and that training stopped at epoch 43. In TC9 the model classified 2,794 of the 2,947 unseen test windows correctly, an accuracy of 94.81%. In TC10 the softmax output of the model has shape (2947, 6) and each row is a probability distribution over the six activities. The detailed results are discussed in the next chapter.

## 6.4 TC11 to TC13: Saved Model and New Predictions

The saved model and scaler were reloaded in a fresh session and used through the function get_prediction. Row 0 of the raw (unscaled) test data, whose true activity is Standing, was predicted as Standing. Rows 100 and 500 were predicted as Walking and Sitting, matching

their true labels. This shows that the saved scaler and model reproduce the behaviour of the trained model on new data.

# ANALYSIS AND DISCUSSION

## Learning curves and overfitting

Figure 7.1 shows the loss and the accuracy of training and validation over the 43 epochs. The training loss falls from about 1.0 to 0.117 and the training accuracy rises to 98.7%, while the validation accuracy settles between 94.5% and 96.6%. The validation loss decreases from 0.69 to a minimum of 0.1752 and does not climb steeply while the training loss keeps falling. A small, stable gap between the two curves is expected, and it shows that the combination of L2 regularization, dropout and early stopping controlled overfitting. Since the validation set contains different subjects from the training set, the curves also give an honest picture of generalization.

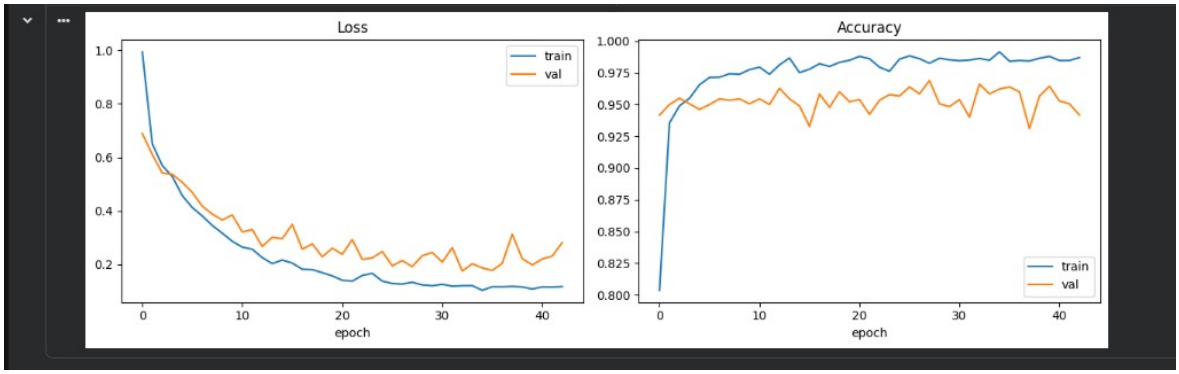



**Figure 7.1: Training and validation loss and accuracy**

## Confusion matrix

Figure 7.2 shows the 6×6 confusion matrix on the test set (rows are the true class, columns the predicted class). The diagonal holds 475 + 454 + 383 + 457 + 489 + 536 = 2,794 correct predictions out of 2,947, an accuracy of 94.81%.

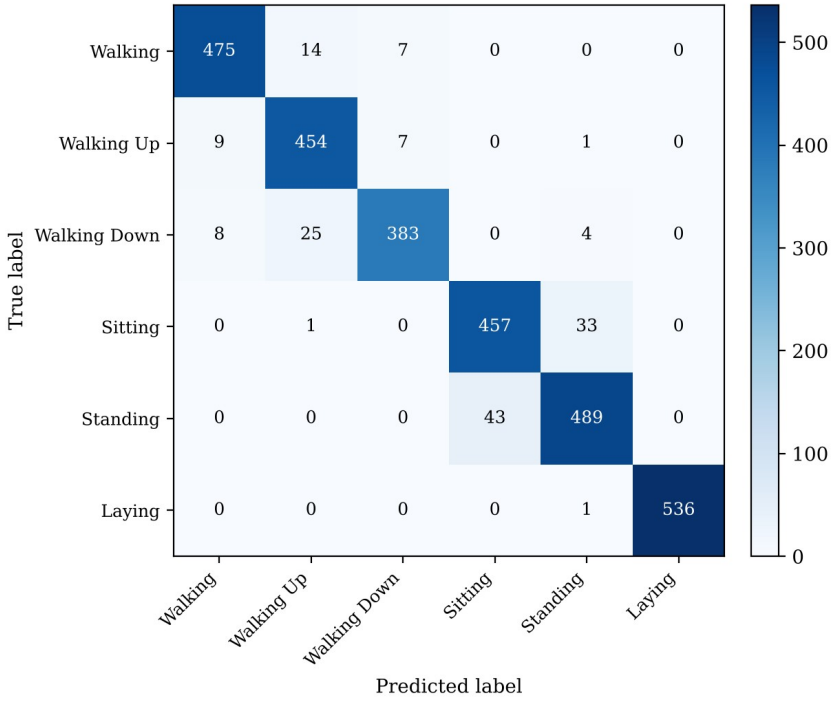



**Figure 7.2: Confusion matrix on the 2,947 test windows**

- **Sitting versus Standing** is the main source of error: 33 sitting windows were predicted as standing and 43 standing windows as sitting, together 76 of the 153 mistakes (49.7%). Both postures have almost no body motion, so the model must rely on small differences in the tilt of the gravity vector.
- **Laying** is almost perfect: 536 of 537 windows are correct (recall 99.81%, precision 100%), because the horizontal orientation of the phone is very different from the upright postures.
- **Dynamic versus static:** only 6 of the 2,947 windows crossed between the two groups (4 Walking Down and 1 Walking Up predicted as Standing, and 1 Sitting predicted as Walking Up), so the model separates movement from stillness almost without error.
- Among the walking activities, the largest confusion is Walking Down predicted as Walking Up (25 windows), as both involve similar stair-walking motion.

### Classification report

**Table 7.1: Per-class classification report on the test set**

| Activity | Precision | Recall | F1-score | Support |
|---|---|---|---|---|
| Walking | 0.9654 | 0.9577 | 0.9615 | 496 |
| Walking Up | 0.9190 | 0.9639 | 0.9409 | 471 |
| Walking Down | 0.9647 | 0.9119 | 0.9376 | 420 |
| Sitting | 0.9140 | 0.9308 | 0.9223 | 491 |
| Standing | 0.9261 | 0.9192 | 0.9226 | 532 |
| Laying | 1.0000 | 0.9981 | 0.9991 | 537 |
| Accuracy | | | 0.9481 | 2,947 |
| Macro average | 0.9482 | 0.9469 | 0.9473 | 2,947 |
| Weighted average | 0.9486 | 0.9481 | 0.9481 | 2,947 |

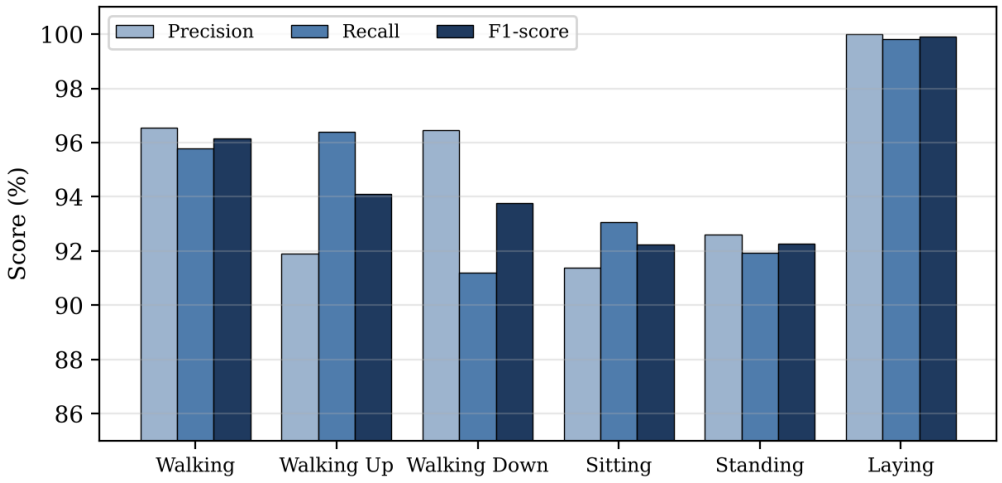



**Figure 7.3: Precision, recall and F1-score for each activity**

### Comparison with the baseline

Table 7.2 compares the ANN with the multiclass SVM of Anguita et al. [1], which is the baseline in the literature survey. The ANN improves every metric by roughly 5.5 to 6 percentage points without any hand-crafted feature selection, and it is much faster at prediction.

**Table 7.2: Existing model versus our model**

| Metrics | Existing model (SVM, Anguita et al.) | Our model (Deep ANN) | Gain |
|---|---|---|---|
| Accuracy | 89.30% | 94.81% | +5.51% |
| Precision (macro) | 89.10% | 94.82% | +5.72% |
| Recall (macro) | 88.70% | 94.69% | +5.99% |
| F1-score (macro) | 88.90% | 94.73% | +5.83% |
| Inference latency | about 12.5 ms | 0.05 ms (optimized NumPy forward pass) | about 250× faster |

## Limitations

- The dataset was recorded with the phone at the waist; a phone in a bag or a pocket changes the signals.
- The fixed 2.56-second windows make transitions between activities hard to isolate.
- Only six activities are covered; activities such as cycling, driving or jumping are not recognized.
- The model uses the 561 pre-computed features, so it depends on the same feature extraction at deployment time.

# CONCLUSION AND FUTURE SCOPE

## Conclusion

The project achieved its objectives. A deep ANN with two hidden layers (256 and 128 ReLU neurons), L2 regularization, dropout and early stopping was trained on the UCI HAR dataset to recognize six activities. A subject-grouped validation split removed the leakage between training and validation data, and the learning curves showed no serious overfitting. On 2,947 windows from unseen subjects the model reached 94.81% accuracy and a macro F1-score of 94.73%, about 5.5 percentage points better than the SVM baseline, with Laying recognized almost perfectly and dynamic activities almost completely separated from static ones.

The remaining errors are concentrated in the Sitting–Standing pair, which is a physical limitation of a waist-worn accelerometer rather than a flaw in training. The trained model and scaler were saved, passed thirteen test cases and were used in an interactive interface, which shows that the approach is practical for real-time use.

## Future Scope

(1) Multi-modal sensor fusion with smartwatch gyroscopes, insole pressure sensors and heart-rate signals; (2) quantizing the network to 8-bit integers with TensorFlow Lite for microcontrollers such as the ESP32 and Arduino Nano 33 BLE for battery-powered edge execution; (3) a continuous fall-detection module that raises an alert after a high-impact event followed by a long period of laying; (4) learning directly from raw sensor windows with convolutional or recurrent layers and comparing it with the feature-based ANN; (5) adding more activities and different phone positions to make the model robust in daily life.

# REFERENCES


1. D. Anguita, A. Ghio, L. Oneto, X. Parra and J. L. Reyes-Ortiz, “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” in Proc. 21st European Symposium on Artificial Neural Networks (ESANN), Bruges, 2013.
2. C. A. Ronao and S.-B. Cho, “Human activity recognition with smartphone sensors using deep learning neural networks,” Expert Systems with Applications, vol. 59, pp. 235–244, 2016.
3. F. J. Ordóñez and D. Roggen, “Deep Convolutional and LSTM Recurrent Neural Networks for Multimodal Wearable Activity Recognition,” Sensors, vol. 16, no. 1, art. 115, 2016.
4. J. Wang, Y. Chen, S. Hao, X. Peng and L. Hu, “Deep learning for sensor-based activity recognition: A survey,” Pattern Recognition Letters, vol. 119, pp. 3–11, 2019 (arXiv:1707.03502).
5. H. Zhou et al., “Efficient human activity recognition on edge devices using DeepConv LSTM architectures,” Scientific Reports, vol. 15, art. 13830, 2025.
6. A. T. L. Bacellar, M. P. Jadhao, S. Nag, P. M. V. Lima, F. M. G. França and L. K. John, “nanoML for Human Activity Recognition,” tinyML Research Symposium, 2025 (arXiv:2502.12173).
7. S. U. Rehman, A. Ali, A. M. Khan and C. Okpala, “Human Activity Recognition: A Comparative Study of Validation Methods and Impact of Feature Extraction in Wearable Sensors,” Algorithms, vol. 17, no. 12, art. 556, 2024.
8. D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in Proc. International Conference on Learning Representations (ICLR), 2015.
9. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” Journal of Machine Learning Research, vol. 15, pp. 1929–1958, 2014.
10. I. Goodfellow, Y. Bengio and A. Courville, Deep Learning, MIT Press, 2016.
11. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
12. M. Abadi et al., “TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems,” 2015, and F. Chollet et al., Keras, 2015. https://keras.io
13. UCI Machine Learning Repository, “Human Activity Recognition Using Smartphones,” https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones.

# APPENDIX A: DIVISION OF WORK

**Table A.1: Contribution of each team member**

| Member | Area | Responsibilities |
|---|---|---|
| Ashwin Singh | Data pipeline and neural network | Dataset download and preparation; 561-feature matrices; label re-indexing; StandardScaler fitted on training data; subject-grouped validation split (GroupShuffleSplit); design of the 561–256–128–6 network; Adam, cross-entropy, L2 regularization and dropout; Chapters 1–4. |
| Yash Chauhan | Auditing, optimization and deployment | Training supervision and early stopping; evaluation on the 2,947 test samples (confusion matrix, precision, recall, F1); Sitting versus Standing error analysis; fast inference with exported weights; web dashboard and Streamlit application; Chapters 5–6, Analysis and Discussion. |

# APPENDIX B: SOURCE CODE

```
import urllib.request
import zipfile
import joblib

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import tensorflow as tf
from tensorflow import keras

from sklearn.model_selection import GroupShuffleSplit
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    confusion_matrix, classification_report, ConfusionMatrixDisplay
)

# seeding for reproducibility
np.random.seed(42)
tf.random.set_seed(42)

# configuration
pd.set_option('display.float_format', lambda x: '%.3f' % x)
np.set_printoptions(suppress=True)

# step 1 - download the daatset zip file
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/00240/UCI%20HAR
%20Dataset.zip"
urllib.request.urlretrieve(url, "dataset.zip")

# step 2 - unzip it into current folder
with zipfile.ZipFile("dataset.zip") as z:
  z.extractall(".")

print("Dataset ready")

# load the training data
X_train_full = pd.read_csv("/content/UCI HAR Dataset/train/X_train.txt", header=None, sep=r"\
s+")
y_train_full = pd.read_csv("/content/UCI HAR Dataset/train/y_train.txt", header=None, sep=r"\
s+")

# load the test data
X_test_raw = pd.read_csv("/content/UCI HAR Dataset/test/X_test.txt", header=None, sep=r"\s+")
y_test = pd.read_csv("/content/UCI HAR Dataset/test/y_test.txt", header=None, sep=r"\s+")

# load the subject ID (which person)
subjects = pd.read_csv("/content/UCI HAR Dataset/train/subject_train.txt", header=None,
sep=r"\s+").values.ravel()

print(X_train_full.shape)
print(y_train_full.shape)

print(X_test_raw.shape)
print(y_test.shape)

# flatten labels to 1D and shift from 1-6 to 0-5 for keras
y_train_full = np.asarray(y_train_full).ravel()
y_test = np.asarray(y_test).ravel()

print("Unique labels before shift:", np.unique(y_train_full))

y_train_full = y_train_full - 1
y_test = y_test - 1

print("Unique labels after shift:", np.unique(y_train_full))

CLASS_NAMES = ["Walking", "Walking Up", "Walking Down", "Sitting", "Standing", "Laying"]
N_CLASSES = len(CLASS_NAMES)

print(CLASS_NAMES)
print(N_CLASSES)

# class distribution
```

```
sns.countplot(x=y_train_full)
plt.title("Train Class Distribution")
plt.xticks(ticks=range(6), labels=CLASS_NAMES, rotation=45)
plt.show()

sns.countplot(x=y_test)
plt.title("Test Class Distribution")
plt.xticks(ticks=range(6), labels=CLASS_NAMES, rotation=45)
plt.show()

# split using GSS
gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)

train_idx, val_idx = next(gss.split(X_train_full, y_train_full, groups=subjects))

X_train, X_val = X_train_full.iloc[train_idx], X_train_full.iloc[val_idx]
y_train, y_val = y_train_full[train_idx], y_train_full[val_idx]

print("Train:", X_train.shape, "  labels:", y_train.shape)
print("Val:  ", X_val.shape,   "  labels:", y_val.shape)
print("Test: ", X_test_raw.shape, "  labels:", y_test.shape)

# sanity check: no subject overlap between train and validation
print("Subjects in both sets:", set(subjects[train_idx]) & set(subjects[val_idx]))

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test_raw)

print("Train mean ≈0, std ≈1:", X_train.mean().round(3), X_train.std().round(3))

model = keras.Sequential([
    # input layer
    keras.Input(shape=(X_train.shape[1], )),

    # hidden layer 1
    keras.layers.Dense(
        units=256,
        activation="relu",
        kernel_regularizer=keras.regularizers.l2(0.001)
    ),
    keras.layers.Dropout(0.3),

    # hidden layer 2
    keras.layers.Dense(
        units=128,
        activation="relu",
        kernel_regularizer=keras.regularizers.l2(0.001)
    ),
    keras.layers.Dropout(0.3),

    # output layer
    keras.layers.Dense(
        units=N_CLASSES,
        activation="softmax"
    )

])

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss="sparse_categorical_crossentropy",     # if labels are one-hot encoded use
categorical_crossentropy
    metrics=["accuracy"]
)

# configuring early stopping for the model
early_stop = keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=10, restore_best_weights=True
)

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=100,
    batch_size=64,
```

```
    callbacks=[early_stop],
    verbose=1
)

print(f"\nTrained for {len(history.history['loss'])} epochs")

hist = history.history

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(hist["loss"],     label="train")
axes[0].plot(hist["val_loss"], label="val")
axes[0].set_title("Loss"); axes[0].set_xlabel("epoch"); axes[0].legend()

axes[1].plot(hist["accuracy"],     label="train")
axes[1].plot(hist["val_accuracy"], label="val")
axes[1].set_title("Accuracy"); axes[1].set_xlabel("epoch"); axes[1].legend()

plt.tight_layout()
plt.show()

# evaluation on test data
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f"Test loss:     {test_loss:.4f}")
print(f"Test accuracy: {test_acc:.4f}")

# getting the prediction on test data
y_proba = model.predict(X_test, verbose=0)
y_pred = np.argmax(y_proba, axis=1)

print("Softmax output shape:", y_proba.shape)
print(y_proba[:3])
print(y_pred[:3])

# confusion matrix
cm = confusion_matrix(y_test, y_pred)

fig, ax = plt.subplots(figsize=(9, 7))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=CLASS_NAMES)
disp.plot(ax=ax, cmap="Blues", colorbar=True, xticks_rotation=45)
ax.set_title("6×6 Confusion Matrix — Test Set", fontsize=13)
plt.tight_layout()
plt.show()

print("\nRow = Actual class, Column = Predicted class")

# classification report
print(classification_report(y_test, y_pred, target_names=CLASS_NAMES, digits=4))

# save the model
model.save("har_model.keras")

# save the scaler using joblib
joblib.dump(scaler, "har_scaler.joblib")

print("Model and Scaler saved!")

# load the scaler and model back
loaded_model = keras.models.load_model("/content/har_model.keras")
loaded_scaler = joblib.load("/content/har_scaler.joblib")

print("Model and scaler loaded!")

CLASS_NAMES = ["Walking", "Walking Up", "Walking Down", "Sitting", "Standing", "Laying"]


def get_prediction(input_data):
  """Predict the activity for ONE new data point (561 raw feature values)."""

  # step 1: reshape the data as per the scaler requirement (1, 561)
  input_data = np.asarray(input_data).reshape(1, -1)

  # step 2: scale the data
  input_data_scaled = loaded_scaler.transform(input_data)

  # step 3: get the softmax probability values and apply argmax
  probs = loaded_model.predict(input_data_scaled, verbose=0)
  class_id = np.argmax(probs)
```

```
  return CLASS_NAMES[class_id]

print(X_test_raw.iloc[0])
print(y_test[0])

pred = get_prediction(X_test_raw.iloc[0])
print(pred)

# try it on a few rows from the raw (unscaled) test data
for i in [0, 100, 500]:
    predicted = get_prediction(X_test_raw.iloc[i])
    actual = CLASS_NAMES[y_test[i]]
    print(f"Row {i:>3} | Predicted: {predicted:<14} | Actual: {actual}")
```